CampusCode

Informe de presentación

Versión: 2.0 **Fecha:** 1 de noviembre de 2026 **Equipo:** CampusCode · Universidad Mayor de Cartagena **Estado:** demostración full-stack funcional, publicada en una URL temporal con HTTPS y con el asistente de IA en modo live verificado.

> **Nota de transparencia:** CampusCode es un proyecto académico de demostración. Las empresas, retos, cuentas, equipos, préstamos, vacantes, pruebas y métricas iniciales son ficticios y están identificados como tales en la interfaz. No representan alianzas, estudiantes, procesos de selección ni resultados reales. La plataforma no ofrece tutorías, tutores, reservas de sesiones ni pagos por tutoría.

Contenido

1. Ficha rápida
2. 1. Resumen ejecutivo
3. 2. Contexto y problema
4. 3. Objetivos
5. 4. La solución: módulos implementados
6. 5. Experiencia por rol
7. 6. Arquitectura y tecnología
8. 7. Seguridad y privacidad
9. 8. Asistente de IA
10. 9. Datos de demostración
11. 10. Pruebas y calidad
12. 11. Despliegue y estado actual
13. 12. Limitaciones y riesgos
14. 13. Hoja de ruta
15. 14. Conclusiones
16. 15. Guion de presentación (5–7 minutos)
17. 16. Preguntas frecuentes de la defensa
18. 17. Anexos

Ficha rápida

Aspecto	Detalle
Producto	Plataforma académica full-stack para practicar, organizar materias, registrar préstamos de PCs y conectar con empresas.
Usuarios	Estudiantes, empresas y administración.
Funciones principales	Banco de ejercicios, asistente de IA, progreso, materias personales, préstamo de computadores, retos, vacantes y pruebas técnicas.
Stack	React 19, TypeScript, Vite, Tailwind CSS, Node.js, Express 5 y SQLite.
IA	Gemini 3.5 Flash mediante API compatible con OpenAI; clave solo en el servidor.
Datos	Semilla de demostración: 7 carreras, 20 materias, 43 ejercicios, 12 computadores, 7 retos, 7 vacantes y 6 pruebas técnicas.
Calidad	14/14 pruebas de integración, TypeScript sin errores, ESLint sin errores y build de producción correcto.
Demo	URL temporal de Cloudflare Tunnel; requiere que el servidor y el túnel estén encendidos.
Dominio propio	`campuscode.com` está registrado en Hover; falta mover el DNS a Cloudflare para publicarlo de forma permanente.

---

1. Resumen ejecutivo

CampusCode es una plataforma académica que reúne en un solo lugar la práctica, el seguimiento del aprendizaje, la organización personal de materias, el préstamo de equipos y el contacto con oportunidades empresariales.

El proyecto nace de una necesidad concreta: el estudiante suele estudiar con materiales dispersos, sin un registro claro de su avance y con pocas oportunidades de enfrentarse a retos reales. Al mismo tiempo, las empresas tienen dificultades para evaluar habilidades prácticas de talento junior y las instituciones gestionan procesos académicos y de préstamo de equipos de forma fragmentada.

CampusCode responde con una aplicación full-stack funcional, no con una maqueta. Hoy se puede recorrer con tres roles, crear materias personales, resolver ejercicios, pedir un computador, postularse a un reto, publicar una vacante, crear una prueba técnica y conversar con un asistente de IA real. La autenticación y las autorizaciones se validan en el servidor.

La entrega es honesta sobre su alcance: es una **demostración funcional** con datos ficticios, publicada en una URL temporal. Todavía no es un despliegue de producción a escala. Para llegar allí faltan el dominio definitivo, PostgreSQL/Supabase, copias de seguridad, correo transaccional, observabilidad y una auditoría de seguridad y accesibilidad.

---

2. Contexto y problema

2.1 El problema del estudiante

- Los contenidos, ejercicios y apuntes viven en plataformas distintas.
- Es difícil saber qué se ha practicado, qué falta y cómo avanza el aprendizaje.
- Muchas herramientas no conectan la práctica con retos o oportunidades reales.
- La organización de materias, horarios y recursos es manual.
- El acceso a equipos de cómputo puede ser un cuello de botella dentro de la institución.

2.2 El problema de la empresa

- Evaluar talento junior solo con hoja de vida es costoso y poco preciso.
- Las pruebas técnicas suelen gestionarse por fuera de la plataforma académica.
- No existe una señal práctica y verificable del trabajo del estudiante.
- Publicar una vacante o un reto requiere canales dispersos.

2.3 El problema de la institución

- Los procesos de préstamo de equipos se llevan en hojas de cálculo o en papel.
- No hay una vista unificada de usuarios, roles, catálogo y actividad.
- La información académica y la empleabilidad se gestionan por separado.

2.4 La oportunidad

Integrar en una sola experiencia:

1. **Aprender:** banco de ejercicios y asistente académico.
2. **Organizar:** materias personales, progreso y objetivos.
3. **Acceder:** préstamo de computadores.
4. **Conectar:** retos, vacantes y pruebas técnicas de empresas.

---

3. Objetivos

3.1 Objetivo general

Construir una plataforma académica full-stack que centralice la práctica, el seguimiento del aprendizaje y la conexión con oportunidades empresariales, con autenticación y autorización verificadas en el servidor.

3.2 Objetivos específicos

- Ofrecer un banco de ejercicios filtrable por carrera, materia, tema, dificultad y tipo de respuesta.
- Permitir que cada estudiante organice sus propias materias y consulte su progreso.
- Gestionar el préstamo y la devolución de computadores por sala.

- Publicar retos, vacantes y pruebas técnicas desde cuentas de empresa.
- Validar roles y propiedad de recursos en cada endpoint privado.
- Integrar un asistente de IA que explique procedimientos sin reemplazar al docente.
- Mantener datos de demostración claramente identificados y evitar métricas inventadas.
- Documentar el estado real, las pruebas y los pendientes de producción.

---

4. La solución: módulos implementados

Módulo	Qué resuelve	Evidencia
Sitio público y catálogo	Presenta la propuesta, carreras, materias, ejercicios, retos y vacantes.	Rutas `/`, `/catalogo`, `/carreras/:slug`, `/empresas`.
Banco de ejercicios	Práctica con filtros, pistas, procedimiento, favoritos y progreso.	43 ejercicios en 7 carreras, 20 materias y 40 temas.
Asistente académico	Explica conceptos paso a paso y conserva historial en cuentas autenticadas.	Modo visitante y conversaciones privadas; IA live con Gemini 3.5 Flash.
Progreso académico	Muestra ejercicios completados, actividad, favoritos, postulaciones y objetivos.	Panel del estudiante y página `/mi-progreso`.
Materias personales	Cada estudiante construye su propio tablero, separado del catálogo global.	`/materias` inicia vacío; alta y baja con validación de propiedad.
Préstamo de computadores	Registra entrada, salida, estado del equipo y disponibilidad.	12 equipos de demostración, préstamos activos e historial.
Retos de empresas	Publicación, postulación, entregables por enlace y cambio de estado.	Panel de empresa y catálogo público de retos.
Portal de reclutamiento	Vacantes y pruebas técnicas con preguntas abiertas o de opción múltiple.	7 vacantes y 6 pruebas de demostración.
Administración	Gestiona usuarios, catálogo, ejercicios, retos y reportes.	Rutas `/admin/*` protegidas por rol.
Autenticación y seguridad	Sesiones, roles, propiedad de recursos y validación de entrada.	bcrypt, JWT en cookie `httpOnly`, CSRF por origen y rate limit.

4.1 Lo que no incluye, por decisión de alcance

- Tutorías, tutores, reservas, horarios o pagos por sesión.
- Un ATS completo con seguimiento de candidaturas.
- Calificación automática de pruebas técnicas desde la cuenta del estudiante.
- Correos transaccionales, carga de archivos o pasarela de pagos.
- Métricas reales de uso o analítica de producto.
- Aplicación móvil nativa.

Estas ausencias son deliberadas y están documentadas para no atribuir al proyecto funciones que no existen.

---

5. Experiencia por rol

Rol	Qué puede hacer
Estudiante	Registrarse, iniciar sesión, explorar el catálogo, resolver ejercicios, guardar favoritos, ver su progreso, crear objetivos, agregar materias personales, pedir y devolver computadores, postularse a retos y conversar con el asistente.
Empresa	Iniciar sesión, administrar su perfil, crear y editar retos propios, revisar postulaciones, publicar vacantes y crear pruebas técnicas con preguntas y puntaje mínimo.
Administración	Ver el resumen del sistema, buscar usuarios, cambiar roles y estado, gestionar carreras y materias, publicar u ocultar ejercicios, moderar retos y revisar reportes.

La protección no depende de ocultar botones: cada endpoint privado valida sesión, rol y, cuando corresponde, la propiedad del recurso.

---

6. Arquitectura y tecnología

6.1 Vista general

```
Navegador
  React 19 + TypeScript + Vite
          │
          │ HTTPS / JSON
          ▼
Servidor Express 5
  ├── API REST con validación y autorización
  ├── Servicio de archivos estáticos (dist/)
  ├── SQLite mediante node:sqlite
  └── Proveedor de IA (Gemini) — solo desde el servidor
```

6.2 Stack

Capa	Tecnología
Interfaz	React 19, TypeScript 5.9, Vite 8, React Router 7
Estilos	Tailwind CSS 4, componentes propios, Phosphor Icons, KaTeX
Servidor	Node.js 24+, Express 5
Base de datos	SQLite con `node:sqlite`, sin ORM
Autenticación	bcrypt (12 rondas) + JWT en cookie `httpOnly`, `SameSite=Lax`
IA	API compatible con OpenAI; Gemini 3.5 Flash
Pruebas	`node:test` + `fetch` contra Express y una base temporal

6.3 Decisiones de diseño

- **Monolito desplegable:** una sola aplicación sirve la API y el build de React. Reduce infraestructura y facilita la demo.
- **SQLite para la demo:** no requiere un servicio externo y permite arrancar con `npm install`. El esquema PostgreSQL queda como referencia de migración.
- **Sin ORM:** consultas SQL parametrizadas y control explícito del esquema.
- **IA en el servidor:** la clave nunca llega al navegador ni al bundle.
- **Datos de demostración deterministas:** `npm run seed:reset` restaura el estado inicial.
- **Autorización en el servidor:** la interfaz puede ocultar acciones, pero la decisión final siempre está en Express.

---

7. Seguridad y privacidad

7.1 Medidas implementadas

- Contraseñas con bcrypt de 12 rondas.
- JWT firmado en cookie `httpOnly`, `SameSite=Lax` y `Secure` en producción.
- Roles `student`, `company` y `admin` comprobados en cada endpoint privado.
- Relectura de la sesión y del estado activo del usuario desde la base.
- Validación centralizada de entrada y consultas SQL parametrizadas.
- Guardia de origen para métodos mutantes (CSRF).
- Rate limit en login, registro y mensajes de IA.
- Cabeceras de seguridad, HSTS y CSP en producción.
- Límite de cuerpo de 120 KB y errores internos sin stack trace.
- Clave de IA solo en variables de entorno del servidor.

7.2 Privacidad

- Los datos iniciales son ficticios y están etiquetados como demostración.
- Las conversaciones autenticadas se guardan en la base local y solo las ve su dueño.
- El modo visitante del asistente no persiste la conversación.
- No se deben introducir datos personales reales en esta demo.
- Antes de un uso institucional se requiere política de retención, consentimiento y proveedor autorizado.

7.3 Límites reconocidos

- El rate limit es en memoria: funciona en una instancia, no en un clúster.
- No se ejecutó un pentest ni un escaneo de dependencias.
- No hay auditoría automatizada de accesibilidad.
- La seguridad de producción completa requiere backups, monitoreo y revisión legal.

---

8. Asistente de IA

8.1 Estado actual

El asistente está conectado a **Gemini 3.5 Flash** mediante el endpoint compatible con OpenAI de Google. La clave se configura en `.env.production` y se usa únicamente en el servidor.

Estado verificado:

```
{
  "configured": true,
  "mode": "live",
  "provider": "openai-compatible",
  "model": "gemini-3.5-flash",
  "message": "Asistente conectado a un proveedor compatible con la API de OpenAI."
}
```

La interfaz muestra **"IA conectada"** y etiqueta cada respuesta como **"Respuesta generada por IA"**.

8.2 Cómo funciona

1. El navegador envía la pregunta a `/api/ai/guest` o a `/api/conversations/:id/messages`.
2. El servidor valida, limita la frecuencia y construye el historial.
3. Si hay clave y `AI_MODE=live`, llama al proveedor externo.
4. Si no hay credenciales, responde con una guía local determinista etiquetada como **"Guía local · no generada por IA"**.
5. Si el proveedor falla, devuelve un error claro; no inventa una respuesta.

8.3 Reglas del asistente

- Responde en español claro y con ejemplos.
- Explica el procedimiento antes del resultado.
- Prioriza el aprendizaje autónomo.
- No inventa fuentes, normas, datos institucionales ni estadísticas.
- Aclara que la orientación en salud, derecho o finanzas es educativa.
- No ofrece tutorías, sesiones, reservas ni reuniones.
- Si no sabe algo, lo dice.

8.4 Costos y límites

- Cada consulta live consume tokens del proveedor.
- Se configuró `AI_MAX_TOKENS=4096` porque Gemini 3.5 Flash usa razonamiento interno y con 900 tokens las respuestas se cortaban.
- Se debe definir un límite de gasto y una política de uso antes de abrir la plataforma a muchos usuarios.
- El asistente puede equivocarse; no reemplaza la orientación docente.

---

9. Datos de demostración

Dato	Cantidad	Tratamiento
Carreras	7	Catálogo de ejemplo
Materias	20	Catálogo de ejemplo
Temas	40	Catálogo de ejemplo
Ejercicios	43	Contenido original de demostración
Retos	7	`is_demo = 1`; algunos en borrador o cerrados
Empresas	5	Ficticias, etiquetadas como demostración
Usuarios	7	Cuentas de demostración
Computadores	12	Equipos ficticios en tres salas
Préstamos	4	Activos e históricos de ejemplo
Vacantes	7	6 abiertas y 1 en borrador
Pruebas técnicas	6	5 publicadas y 1 en borrador

9.1 Cuentas de demostración

Rol	Correo	Contraseña
Estudiante	`estudiante@campuscode.co`	`Campus123!`
Empresa	`empresa@campuscode.co`	`Campus123!`
Administración	`admin@campuscode.co`	Contraseña rotada; no se incluye en este informe

Las empresas adicionales (`aurora@`, `legal@`, `pacifico@`, `eco@campuscode.co`) usan la misma contraseña de demostración.

10. Pruebas y calidad

10.1 Resultados verificados

Comprobación	Resultado
`npm run typecheck`	Correcto, sin errores
`npm run lint`	0 errores, 19 advertencias de estilo
`npm test`	14/14 pruebas aprobadas
`npm run build`	Correcto; bundle principal de 1,27 MB con advertencia de tamaño
Rutas públicas y paneles	Verificados en Chromium sin errores de consola

Comprobación	Resultado
Responsive	390 px, 820 px y 1440 px sin scroll horizontal de página
IA live	Verificada con Gemini 3.5 Flash en la URL pública
Materias personales	Alta, duplicado bloqueado y baja verificados
Préstamo de PCs	Registro, préstamo activo, cierre y liberación verificados
Vacantes y pruebas	CRUD de empresa y vista pública sin respuestas correctas
Autorización	Roles y propiedad de recursos verificados en el servidor

10.2 Pruebas de integración

La suite cubre:

1. Contrato live de proveedor compatible con OpenAI.
2. Contrato live de la API nativa de Anthropic.
3. Salud pública y catálogo.
4. Login, cookie `httpOnly` y sesión.
5. Registro, roles y cambios de estado del administrador.
6. Autorización por rol.
7. Favoritos, progreso y objetivos.
8. Postulación a reto y duplicados.
9. Propiedad de retos entre empresas.
10. Materias personales y préstamo de computadores.
11. Vacantes, pruebas y gestión por empresa.
12. Administración de catálogo, ejercicios y reportes.
13. Guía local sin credenciales.
14. Protección de origen para peticiones mutantes.

10.3 Lo que no se probó

- Migración real a PostgreSQL/Supabase.
- Carga, estrés y concurrencia.
- Matriz completa de navegadores y dispositivos físicos.
- Auditoría automatizada de accesibilidad.
- Pentest o escaneo de dependencias.
- Rate limit distribuido entre varias instancias.
- Otros proveedores de IA reales distintos de Gemini.

---

11. Despliegue y estado actual

11.1 Desarrollo local

```
npm install
npm run seed:reset
npm run dev
```

- API: `http://localhost:4000`
- Interfaz: `http://localhost:5173`

11.2 Producción local

```
npm run build
npm run start:production
```

Express sirve `dist/` y aplica `NODE_ENV=production`, cookie `Secure`, HSTS y CSP.

11.3 URL pública temporal

Mientras el DNS de `campuscode.com` no esté activo en Cloudflare, la demo se publica con un túnel rápido:

```
https://mirrors-sherman-interpretation-web.trycloudflare.com
```

Limitaciones: la URL cambia si se reinicia el túnel y solo funciona mientras este PC, el servidor y `cloudflared` estén encendidos. No sustituye al dominio propio.

11.4 Dominio propio

`campuscode.com` está registrado en Hover y todavía usa los nameservers de Hover. Para publicarlo de forma permanente con Cloudflare Tunnel falta:

1. Crear el sitio en Cloudflare.
2. Reemplazar los nameservers en Hover.
3. Esperar el estado **Active**.
4. Crear el túnel y enrutar `campuscode.com` y `www`.
5. Verificar HTTPS y el health check.

---

12. Limitaciones y riesgos

Limitación	Impacto	Siguiente paso
Datos ficticios	No permite conclusiones sobre usuarios reales.	Pilotear con usuarios y datos consentidos.

Limitación	Impacto	Siguiente paso
SQLite en una instancia	No escala horizontalmente.	Implementar adaptador PostgreSQL y probar la migración.
Rate limit en memoria	Se reinicia y no se comparte entre instancias.	Usar Redis, Upstash o una tabla compartida.
URL temporal	Depende de este PC y cambia al reiniciar.	Activar el dominio en Cloudflare.
Sin correo transaccional	No hay verificación ni recuperación de contraseña.	Integrar proveedor de correo.
Sin carga de archivos	Los entregables son enlaces.	Definir almacenamiento y validación.
Sin ATS	No hay seguimiento completo de candidaturas.	Diseñar el flujo con empresas.
Sin calificación automática	Las pruebas técnicas no se corrigen solas.	Implementar motor de evaluación.
IA puede equivocarse	Riesgo de respuestas incorrectas.	Revisión humana, moderación y política de uso.
Costo de IA	Cada consulta consume tokens.	Límites de gasto y cuotas por usuario.
Bundle de 1,27 MB	Carga inicial más pesada.	División por rutas con `import()`.
Sin auditoría de accesibilidad	Puede haber barreras no detectadas.	Ejecutar axe y pruebas con usuarios.

13. Hoja de ruta

Corto plazo (0–1 mes)

- Activar `campuscode.com` con Cloudflare.
- Programar copias de seguridad y probar la restauración.
- Rotar contraseñas de demostración antes de abrir el sitio.
- Configurar límites de gasto y cuotas para la IA.
- Añadir monitoreo básico y alertas.

Mediano plazo (1–3 meses)

- Implementar el adaptador PostgreSQL/Supabase y migrar la semilla.
- Añadir correo transaccional y recuperación de contraseña.
- Rate limit compartido.
- Calificación automática de pruebas técnicas.
- Onboarding de empresas con un flujo revisado.

Largo plazo (3–6 meses)

- Analítica de producto con consentimiento.
- Auditoría de accesibilidad y matriz de navegadores.
- PWA o aplicación móvil.

- Integraciones institucionales.
- Piloto con estudiantes y empresas reales.

---

14. Conclusiones

CampusCode demuestra que es posible integrar en una sola aplicación la práctica académica, el progreso personal, la gestión de recursos y la conexión con empresas, con una base técnica verificable y una experiencia usable en móvil y escritorio.

El valor de la entrega está en tres cosas:

1. **Funciona:** se puede recorrer con tres roles y ejecutar flujos completos.
2. **Es verificable:** hay pruebas, documentación y evidencia reproducible.
3. **Es honesta:** distingue datos ficticios, modo local y modo live, y declara lo que falta.

Todavía no es un producto listo para producción a escala. Es una **demostración full-stack sólida**, apta para presentación académica, validación con usuarios y una siguiente etapa de migración a infraestructura real.

---

15. Guion de presentación (5–7 minutos)

Diapositiva 1 — Gancho (30 s)

> "Un estudiante puede tener apuntes, ejercicios, materias y oportunidades en cinco lugares distintos. CampusCode los une en una sola experiencia."

Diapositiva 2 — Problema (45 s)

- Práctica dispersa.
- Progreso invisible.
- Gestión manual de equipos.
- Poca conexión entre aprendizaje y empresa.

Diapositiva 3 — Solución (1 min)

Mostrar los cuatro pilares: **aprender, organizar, acceder y conectar**.

Diapositiva 4 — Demo (2 min)

Recorrido sugerido:

1. Landing y catálogo.
2. Ejercicio con pista y procedimiento.
3. Asistente con una pregunta académica.

4. Panel de progreso y materias personales.
5. Préstamo de un computador.
6. Reto, vacante y prueba técnica.
7. Panel de administración.

Diapositiva 5 — Arquitectura y seguridad (45 s)

- React + Express + SQLite.
- Tres roles y autorización en el servidor.
- Clave de IA solo en el servidor.

Diapositiva 6 — IA (45 s)

- Gemini 3.5 Flash en modo live.
- Guía local cuando no hay credenciales.
- El asistente explica, no reemplaza al docente.

Diapositiva 7 — Resultados y hoja de ruta (45 s)

- 14/14 pruebas.
- Responsive 390/820/1440.
- Próximo paso: dominio, PostgreSQL y backups.

Diapositiva 8 — Cierre (15 s)

> "CampusCode no promete tutorías ni métricas inventadas. Promete una base real para aprender, organizar y conectar."

---

16. Preguntas frecuentes de la defensa

¿La IA es real? Sí. Está conectada a Gemini 3.5 Flash y se verificó una consulta real desde la URL pública. La clave vive solo en el servidor. Sin clave, la app muestra una guía local etiquetada.

¿Las empresas y vacantes son reales? No. Son datos de demostración identificados como tales. No representan alianzas ni procesos de selección.

¿Ofrecen tutorías? No. CampusCode no tiene tutores, reservas ni pagos por sesión. El acompañamiento se limita al banco de ejercicios, el asistente y los retos.

¿Está listo para producción? No a escala. Es una demo funcional. Faltan dominio permanente, PostgreSQL, backups, correo, observabilidad y auditoría de seguridad.

¿Por qué SQLite? Porque permite una demo reproducible sin infraestructura externa. El esquema PostgreSQL está preparado como referencia, pero la migración requiere un adaptador y pruebas.

¿Cómo protegen los datos? bcrypt, JWT en cookie `httpOnly`, roles, propiedad de recursos, validación de entrada, SQL parametrizado, CSRF por origen y rate limit.

¿Qué pasa si se cae el PC? La URL temporal deja de funcionar. Para disponibilidad permanente se necesita un dominio con túnel en un equipo siempre encendido o un VPS.

---

17. Anexos

17.1 Rutas principales

Ruta	Descripción
`/`	Landing y propuesta de valor
`/catalogo`	Catálogo de carreras y materias
`/ejercicios`	Banco de ejercicios
`/asistente`	Asistente académico
`/retos`	Retos de empresas
`/empresas`	Portal de empresas, vacantes y pruebas
`/dashboard`	Panel del estudiante
`/materias`	Materias personales
`/prestamo-pcs`	Préstamo de computadores
`/empresa`	Panel de empresa
`/admin`	Panel de administración

17.2 Comandos útiles

```
npm install
npm run seed:reset
npm run dev
npm run typecheck
npm run lint
npm test
npm run build
npm run start:production
```

17.3 Documentación de referencia

- `README.md`: instalación, rutas, API y configuración.
- `docs/ESTADO.md`: estado y pendientes.
- `docs/PRUEBAS.md`: comandos, resultados y evidencia.
- `docs/IA.md`: configuración del asistente.

- `docs/BASE-DE-DATOS.md` : SQLite, PostgreSQL y Supabase.
- `docs/PRODUCCION.md` : checklist de despliegue.
- `docs/DOMINIO.md` : Cloudflare Tunnel y dominio.
- `docs/evidence/` : capturas de la interfaz.

Fin del informe.